



## Chapter 6: Formal Relational Query Languages

### - Relational Algebra -

Database System Concepts, 6<sup>th</sup> Ed.

©Silberschatz, Korth and Sudarshan (revised by Sangwoo Park)

See [www.db-book.com](http://www.db-book.com) for conditions on re-use

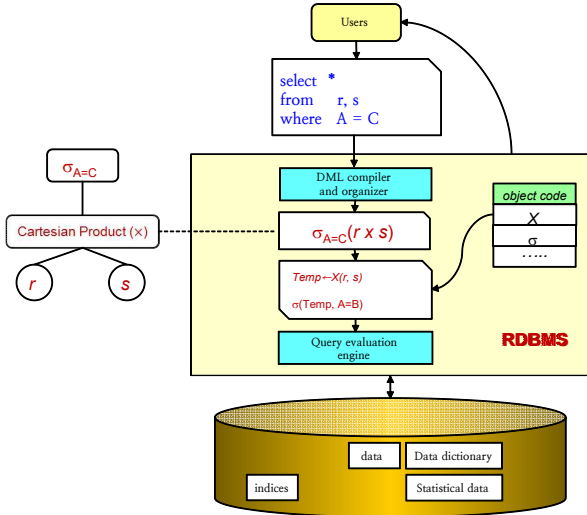


# Relational Algebra

- n Procedural language
- n Six basic operators
  - | select:  $\sigma$
  - | project:  $\Pi$
  - | union:  $\cup$
  - | set difference:  $-$
  - | Cartesian product:  $\times$
  - | rename:  $\rho$
- n Additional operators
  - | Set intersection
  - | Natural join
  - | Assignment
  - | Outer Join
- ◆ Extended relational algebra operators
  - Generalized Projection
  - Aggregate Functions
- n The operators take one or two relations as inputs and produce a new relation as a result.
  - |  $op\ r1$
  - |  $r1\ op\ r2$



# Query Processing Procedure



| r        |   | s        |    |   |
|----------|---|----------|----|---|
| A        | B | C        | D  | E |
| $\alpha$ | 1 | $\alpha$ | 10 | a |
| $\beta$  | 2 | $\beta$  | 10 | a |
|          |   | $\beta$  | 20 | b |
|          |   | $\gamma$ | 10 | b |





# An Implementation of R.A. (in TAJO<sup>TM</sup>)

apache / tajo

Watch

20

Star

111

Fork

100

Code

Pull requests 34

Projects 0

Insights

Tree: 4f3c386eba

tajo / tajo-algebra / src / main / java / org / apache / tajo / algebra /

Create new file

Find file

History



dongjinleekr and jihoonson TAJO-2165: Add 'ALTER TABLE UNSET PROPERTY' statement to Tajo DDL. ...

Latest commit fab36f9 on 11 Jul 2016

Aggregation.java

TAJO-1766: Improve the performance of cross join.

4 years ago

BinaryOperator.java

TAJO-774: Implement logical plan part and physical executor for windo...

5 years ago

Join.java

TAJO-1574: Fix NPE on natural join.

4 years ago

JoinType.java

TAJO-142: Implement hash anti-join operator. (hyunsik)

6 years ago

NamedExpr.java

TAJO-774: Implement logical plan part and physical executor for windo...

5 years ago

Projection.java

TAJO-1947: Change Projection::setNamedExprs and getNamedExprs to set ...

3 years ago

Selection.java

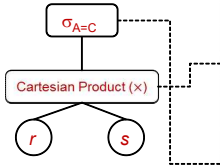
TAJO-774: Implement logical plan part and physical executor for windo...

5 years ago

SetOperation.java

TAJO-774: Implement logical plan part and physical executor for windo...

5 years ago





# Select Operation

n Notation:  $\sigma_p(r)$  - operator to select rows satisfying the condition

n  $p$  is called the **selection predicate**

n Defined as:

t라는 행이 relation r에 들어 있고

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

그 행 t가 우리가 정한 조건 p를 만족한다

Where  $p$  is a formula in propositional calculus

consisting of **terms** connected by :  $\wedge$  (**and**),  $\vee$  (**or**),  $\neg$  (**not**)

n Each **term** is one of:

$\langle \text{attribute} \rangle \text{ op } \{ \langle \text{attribute} \rangle \text{ or } \langle \text{constant} \rangle \}$

**salary > 50000**

**A = B**

where  $op$  is one of:  $=, \neq, >, \geq, <, \leq$

n **Select : operator to select** rows satisfying the given condition (selection predicate)

n Example of selection:

$$\sigma_{\text{dept\_name}=\text{"Physics"}}(\text{instructor})$$

relation r에 들어 있는 행 중에서 조건 p를  
만족하는 행들만 모아라



## Select Operation – Example

◆ Relation r

| A        | B        | C  | D  |
|----------|----------|----|----|
| $\alpha$ | $\alpha$ | 1  | 7  |
| $\alpha$ | $\beta$  | 5  | 7  |
| $\beta$  | $\beta$  | 12 | 3  |
| $\beta$  | $\beta$  | 23 | 10 |

■  $\sigma_{A=B \wedge D > 5}(r)$

| A        | B        | C  | D  |
|----------|----------|----|----|
| $\alpha$ | $\alpha$ | 1  | 7  |
| $\beta$  | $\beta$  | 23 | 10 |

SQL

```
select A, B, C, D
from r
where A=B and D>5;
```



# An Implementation of SELECTION Operator

tajo / tajo-core / src / main / java / org / apache / tajo / engine / planner / physical / SelectionExec.java

ghoonan TAJO-1344: Python UDF support. (ghoonan)

a745385 on 18 Apr 2015

4 contributors

62 lines (52 cloc) 3.9 KB

Raw | Blame | History

```
1 /**
2  * Licensed to the Apache Software Foundation (ASF) under one
3  * or more contributor license agreements. See the NOTICE file
4  * distributed with this work for additional information
5  * regarding copyright ownership. The ASF licenses this file
6  * to you under the Apache License, Version 2.0 (the
7  * "License"); you may not use this file except in compliance
8  * with the License. You may obtain a copy of the License at
9  *
10  * http://www.apache.org/licenses/LICENSE-2.0
11  *
12  * Unless required by applicable law or agreed to in writing, software
13  * distributed under the License is distributed on an "AS IS" BASIS,
14  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
15  * See the License for the specific language governing permissions and
16  * limitations under the License.
17  */
18
19 package org.apache.tajo.engine.planner.physical;
20
21 import org.apache.tajo.engine.codegen.CompilationError;
22 import org.apache.tajo.plan.expr.TvalNode;
23 import org.apache.tajo.plan.logical.SelectionNode;
24 import org.apache.tajo.storage.Tuple;
25 import org.apache.tajo.worker.TaskAttemptContext;
26
27 import java.io.IOException;
```

```
29 public class SelectionExec extends UnaryPhysicalExec {
30     private EvalNode qual;
31
32     public SelectionExec(TaskAttemptContext context,
33                         SelectionNode plan,
34                         PhysicalExec child) {
35         super(context, plan.getInSchema(), plan.getOutSchema(), child);
36         this.qual = plan.getQual();
37     }
38
39     @Override
40     public void init() throws IOException {
41         super.init();
42         qual.bind(context.getEvalContext(), inSchema);
43     }
44
45     @Override
46     public void compile() throws CompilationError {
47         qual = context.getPrecompiledEval(inSchema, qual);
48     }
49
50     @Override
51     public Tuple next() throws IOException {
52         Tuple tuple;
53         while (!context.isStopped() && (tuple = child.next()) != null) {
54             if (qual.eval(tuple).isTrue()) {
55                 return tuple;
56             }
57         }
58
59         return null;
60     }
61 }
```



# Project Operation

n Notation:

$$\Pi_{A_1, A_2, \dots, A_k}(r)$$

where  $A_1, A_2$  are attribute names and  $r$  is a relation name.

- n The result is defined as the relation of  $k$  columns obtained by erasing the columns that are not listed
- n Duplicate rows removed from result, since relations are sets
- n Example: To eliminate the *dept\_name* attribute of *instructor*

$$\Pi_{ID, name, salary}(instructor)$$

student

| name | dept | year |
|------|------|------|
| 김철수  | 컴퓨터  | 1    |
| 이영희  | 컴퓨터  | 2    |
| 김철수  | 전자   | 1    |

$$\Pi_{dept}(student)$$

| dept |
|------|
| 컴퓨터  |
| 전자   |

$$\Pi_{name, year}(student)$$

| name | year |
|------|------|
| 김철수  | 1    |
| 이영희  | 2    |



# Project Operation – Example

n Relation  $r$ :

| A        | B  | C |
|----------|----|---|
| $\alpha$ | 10 | 1 |
| $\alpha$ | 20 | 1 |
| $\beta$  | 30 | 1 |
| $\beta$  | 40 | 2 |

♦  $\Pi_{A,C}(r)$

| A        | C |
|----------|---|
| $\alpha$ | 1 |
| $\alpha$ | 1 |
| $\beta$  | 1 |
| $\beta$  | 2 |

=

| A        | C |
|----------|---|
| $\alpha$ | 1 |
| $\beta$  | 1 |
| $\beta$  | 2 |

SQL

```
select A, C  
from r;
```

```
select distinct (A, C)  
from r;
```



# Union Operation

n Notation:  $r \cup s$

n Defined as:

$$r \cup s = \{t \mid$$

| CH(1) INT |      | CH(1) INT |      |
|-----------|------|-----------|------|
| SID       | SAGE | EID       | EAGE |
| $\alpha$  | 1    | $\alpha$  | 2    |
| $\alpha$  | 2    | $\beta$   | 3    |
| $r$       |      | $s$       |      |

n For  $r \cup s$  to be valid.

1.  $r, s$  must have the **same arity** (same number of attributes)
2. The **attribute domains** must be **compatible** (example: 2<sup>nd</sup> column of  $r$  deals with the same type of values as does the 2<sup>nd</sup> column of  $s$ )

n Example: to find all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or in both

$\Pi_{course\_id} (\sigma_{semester="Fall" \wedge year=2009} (section)) \cup$

$\Pi_{course\_id} (\sigma_{semester="Spring" \wedge year=2010} (section))$





## Union Operation – Example

Relations  $r, s$ :

| A        | B |
|----------|---|
| $\alpha$ | 1 |
| $\alpha$ | 2 |
| $\beta$  | 1 |

$r$

| A        | B |
|----------|---|
| $\alpha$ | 2 |
| $\beta$  | 3 |

$s$

◆  $r \cup s$ :

| A        | B |
|----------|---|
| $\alpha$ | 1 |
| $\alpha$ | 2 |
| $\beta$  | 1 |
| $\beta$  | 3 |

SQL

```
(select  A, B from  r)
union
(select  A, B from  s)
```



# Set Difference Operation

- n Notation  $r - s$  relation  $r$ 에는 들어 있지만 relation  $s$ 에는 들어 있지 않은 tuple만 남겨라.
- n Defined as:  
$$r - s = \{t \mid t \in r \text{ and } t \notin s\}$$
- n Set differences must be taken between **compatible** relations.
  - |  $r$  and  $s$  must have the **same** arity
  - | attribute domains of  $r$  and  $s$  must be **compatible**
- n Example: to find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\Pi_{\text{course\_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009}(\text{section})) - \Pi_{\text{course\_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010}(\text{section}))$$



# Set difference of two relations

n Relations  $r, s$ :

| A        | B |
|----------|---|
| $\alpha$ | 1 |
| $\alpha$ | 2 |
| $\beta$  | 1 |

$r$

| A        | B |
|----------|---|
| $\alpha$ | 2 |
| $\beta$  | 3 |

$s$

♦  $r - s$ :

| A        | B |
|----------|---|
| $\alpha$ | 1 |
| $\beta$  | 1 |

SQL

```
(select A, B from r)
except
(select A, B from s)
```



# Cartesian-Product Operation

- n Notation  $r \times s$  relation  $r$ 의 각 tuple과 relation  $s$ 의 각 tuple을 가능한 모든 경우로 조합하는 것
- n Defined as:

$$r \times s = \{t \ q \mid t \in r \text{ and } q \in s\}$$

- n Assume that attributes of  $r(R)$  and  $s(S)$  are disjoint. (That is,  $R \cap S = \emptyset$ ).
- n If attributes of  $r(R)$  and  $s(S)$  are not disjoint, then renaming must be used.



# Cartesian-Product Operation – Example

◆ Relations  $r$ ,  $s$ :

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\beta$  | 2   |

$r$

| $C$      | $D$ | $E$ |
|----------|-----|-----|
| $\alpha$ | 10  | a   |
| $\beta$  | 10  | a   |
| $\beta$  | 20  | b   |
| $\gamma$ | 10  | b   |

$s$

◆  $r \times s$ :

| $A$      | $B$ | $C$      | $D$ | $E$ |
|----------|-----|----------|-----|-----|
| $\alpha$ | 1   | $\alpha$ | 10  | a   |
| $\alpha$ | 1   | $\beta$  | 10  | a   |
| $\alpha$ | 1   | $\beta$  | 20  | b   |
| $\alpha$ | 1   | $\gamma$ | 10  | b   |
| $\beta$  | 2   | $\alpha$ | 10  | a   |
| $\beta$  | 2   | $\beta$  | 10  | a   |
| $\beta$  | 2   | $\beta$  | 20  | b   |
| $\beta$  | 2   | $\gamma$ | 10  | b   |

SQL

```
select *  
from r, s
```



# Rename Operation

- n Allows us to name, and therefore to refer to, the results of relational-algebra expressions.
- n Allows us to refer to a relation by more than one name.
- n Example:

$$\rho_x(E)$$

returns the expression  $E$  under the name  $X$

- n If a relational-algebra expression  $E$  has arity  $n$ , then

$$\rho_{x(A_1, A_2, \dots, A_n)}(E)$$

returns the result of expression  $E$  under the name  $X$ , and with the attributes renamed to  $A_1, A_2, \dots, A_n$ .

$r$

| A        | B  | C |
|----------|----|---|
| $\alpha$ | 10 | a |
| $\beta$  | 10 | a |
| $\beta$  | 20 | b |
| $\gamma$ | 10 | b |



$t$

| M        | N  | O |
|----------|----|---|
| $\beta$  | 20 | b |
| $\gamma$ | 10 | b |

$\sigma_{A=\gamma \vee B > 10}(r)$

| A        | B  | C |
|----------|----|---|
| $\beta$  | 20 | b |
| $\gamma$ | 10 | b |



$$\rho_{t(M, N, O)}(\sigma_{A=\gamma \vee B > 10}(r))$$





# Composition of Operations

- Can build expressions using multiple operations
- Example:  $\sigma_{A=C}(r \times s)$

$r \times s$

| A        | B | C        | D  | E |
|----------|---|----------|----|---|
| $\alpha$ | 1 | $\alpha$ | 10 | a |
| $\alpha$ | 1 | $\beta$  | 10 | a |
| $\alpha$ | 1 | $\beta$  | 20 | b |
| $\alpha$ | 1 | $\gamma$ | 10 | b |
| $\beta$  | 2 | $\alpha$ | 10 | a |
| $\beta$  | 2 | $\beta$  | 10 | a |
| $\beta$  | 2 | $\beta$  | 20 | b |
| $\beta$  | 2 | $\gamma$ | 10 | b |

$\sigma_{A=C}(r \times s)$

| A        | B | C        | D  | E |
|----------|---|----------|----|---|
| $\alpha$ | 1 | $\alpha$ | 10 | a |
| $\beta$  | 2 | $\beta$  | 10 | a |
| $\beta$  | 2 | $\beta$  | 20 | b |

| A        | B |
|----------|---|
| $\alpha$ | 1 |
| $\beta$  | 2 |

$r$

| C        | D  | E |
|----------|----|---|
| $\alpha$ | 10 | a |
| $\beta$  | 10 | a |
| $\beta$  | 20 | b |
| $\gamma$ | 10 | b |

$s$

| A        | B | C        | D  | E |
|----------|---|----------|----|---|
| $\alpha$ | 1 | $\alpha$ | 10 | a |
| $\alpha$ | 1 | $\beta$  | 10 | a |
| $\alpha$ | 1 | $\beta$  | 20 | b |
| $\alpha$ | 1 | $\gamma$ | 10 | b |
| $\beta$  | 2 | $\alpha$ | 10 | a |
| $\beta$  | 2 | $\beta$  | 10 | a |
| $\beta$  | 2 | $\beta$  | 20 | b |
| $\beta$  | 2 | $\gamma$ | 10 | b |



# Example Queries

- n Find the names of all instructors in the Physics department, along with the *course\_id* of all courses they have taught

- Query 1

$$\Pi_{Instructor.name, course\_id} (\sigma_{dept\_name="Physics"} ( \sigma_{instructor.ID=teaches.ID} (instructor \times teaches)))$$



- Query 2

$$\Pi_{instructor.name, course\_id} (\sigma_{instructor.ID=teaches.ID} ( \sigma_{dept\_name="Physics"} (instructor) \times teaches))$$





# Formal Definition of the Relational Algebra

- n A basic expression in the relational algebra consists of either one of the following:
  - | A relation in the database
  - | A constant relation
- n Let  $E_1$  and  $E_2$  be **relational-algebra expressions**; the following are all relational-algebra expressions:
  - |  $E_1 \cup E_2$
  - |  $E_1 - E_2$
  - |  $E_1 \times E_2$
  - |  $\sigma_P(E_1)$ ,  $P$  is a predicate on attributes in  $E_1$
  - |  $\Pi_S(E_1)$ ,  $S$  is a list consisting of some of the attributes in  $E_1$
  - |  $\rho_X(E_1)$ ,  $x$  is the new name for the result of  $E_1$



# Additional Operations

We define additional operations that do not add any power to the relational algebra, but that simplify common queries.

- n Set intersection
- n Natural join
- n Assignment
- n Outer join



# Set-Intersection Operation

- n Notation:  $r \cap s$
  - n Defined as:
  - n  $r \cap s = \{ t \mid t \in r \text{ and } t \in s \}$
  - n Assume:
    - |  $r, s$  have the *same arity*
    - | attributes of  $r$  and  $s$  are compatible
  - n Note:  $r \cap s = r - (r - s)$
- relation  $r$ 에도 있고, relation  $s$ 에도 있는 tuple만 가져와라.

$r(1, 2, 3), \quad s(1, 2, 4)$

$$\begin{aligned} r \cap s &= r - (r - s) = (1, 2, 3) - ((1, 2, 3) - (1, 2, 4)) \\ &= (1, 2, 3) - (3) \\ &= (1, 2) \end{aligned}$$



# Set-Intersection Operation – Example

n Relation  $r, s$ :

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 1   |
| $\alpha$ | 2   |
| $\beta$  | 1   |

$r$

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 2   |
| $\beta$  | 3   |

$s$

n  $r \cap s$

| $A$      | $B$ |
|----------|-----|
| $\alpha$ | 2   |



# Natural-Join Operation

◆ Notation:  $\bowtie$  s

n Let  $r$  and  $s$  be relations on schemas  $R$  and  $S$  respectively.  
Then,  $r \bowtie s$  is a relation on schema  $R \cup S$  obtained as follows:

- | Consider each pair of tuples  $t_r$  from  $r$  and  $t_s$  from  $s$ .
- | If  $t_r$  and  $t_s$  have the same value on each of the attributes in  $R \cap S$ , add a tuple  $t$  to the result, where
  - ▶  $t$  has the same value as  $t_r$  on  $r$
  - ▶  $t$  has the same value as  $t_s$  on  $s$

n Example:

$R = (A, B, C, D)$

$S = (E, B, D)$

| Result schema =  $(A, B, C, D, E)$

|  $r \bowtie s$  is defined as:

$$\Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$$

If  $R \cap S = \emptyset$ ,  
then  $r \bowtie s = r \times s$



# Natural Join Example

n Relations r, s:

| A        | B | C        | D |
|----------|---|----------|---|
| $\alpha$ | 1 | $\alpha$ | a |
| $\beta$  | 2 | $\gamma$ | a |
| $\gamma$ | 4 | $\beta$  | b |
| $\alpha$ | 1 | $\gamma$ | a |
| $\delta$ | 2 | $\beta$  | b |

r

| B | D | E          |
|---|---|------------|
| 1 | a | $\alpha$   |
| 3 | a | $\beta$    |
| 1 | a | $\gamma$   |
| 2 | b | $\delta$   |
| 3 | b | $\epsilon$ |

s

♦  $r \bowtie s$

| A        | B | C        | D | E        |
|----------|---|----------|---|----------|
| $\alpha$ | 1 | $\alpha$ | a | $\alpha$ |
| $\alpha$ | 1 | $\alpha$ | a | $\gamma$ |
| $\alpha$ | 1 | $\gamma$ | a | $\alpha$ |
| $\alpha$ | 1 | $\gamma$ | a | $\gamma$ |
| $\delta$ | 2 | $\beta$  | b | $\delta$ |



# Natural Join and Theta Join

- Find the names of all instructors in the Comp. Sci. department together with the course titles of all the courses that the instructors teach

$\Pi_{name, title} (\sigma_{dept\_name="Comp. Sci."} (instructor \bowtie teaches \bowtie course))$

n Natural join is commutative

| name       | title      | dept_name  | course_id | title      |
|------------|------------|------------|-----------|------------|
| Srinivasan | DB         | Comp. Sci. | CS-101    | DB         |
| Wu         | Investment | Finance    | FIN-201   | Investment |

n Natural join is commutative

n 교수 정보 + 그 교수가 가르치는 과목 정보  
The natural join of instructor and teaches is defined as

$$r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)$$

$$\checkmark r \bowtie s = \Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$$

instructor

| ID    | name       | dept_name  |
|-------|------------|------------|
| 10101 | Srinivasan | Comp. Sci. |
| 12121 | Wu         | Finance    |
| 15151 | Mozart     | Music      |

teaches

| ID    | course_id |
|-------|-----------|
| 10101 | CS-101    |
| 12121 | FIN-201   |
| 76766 | BIO-101   |

course

| course_id | title             | dept_name  |
|-----------|-------------------|------------|
| CS-101    | DB                | Comp. Sci. |
| FIN-201   | Investment        | Finance    |
| BIO-101   | Intro. to Biology | Biology    |



# Assignment Operation

- n The assignment operation ( $\leftarrow$ ) provides a convenient way to express complex queries.
  - | Write query as a sequential program consisting of
    - ▶ a series of assignments
    - ▶ followed by an expression whose value is displayed as a result of the query.
  - | Assignment must always be made to a temporary relation variable.

$$\Pi_{R-S}(r) - \Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$



```
temp1  $\leftarrow \Pi_{R-S}(r)$   
temp2  $\leftarrow \Pi_{R-S}((temp1 \times s) - \Pi_{R-S,S}(r))$   
result  $\leftarrow temp1 - temp2$ 
```



# Outer Join

- n An extension of the join operation that avoids loss of information.
- n Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- n Uses *null* values:
  - | **null** signifies that the value is unknown or does not exist
  - | All comparisons involving *null* are (roughly speaking) **false** by definition.
    - ▶ We shall study precise meaning of comparisons with nulls later





# Outer Join – Example

n Relation *instructor*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> |
|-----------|-------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       |
| 12121     | Wu          | Finance          |
| 15151     | Mozart      | Music            |

◆ Relation *teaches*

| <i>ID</i> | <i>course_id</i> |
|-----------|------------------|
| 10101     | CS-101           |
| 12121     | FIN-201          |
| 76766     | BIO-101          |





# Outer Join – Example

n Natural Join

*instructor* ⋈ *teaches*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>course_id</i> |
|-----------|-------------|------------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       | CS-101           |
| 12121     | Wu          | Finance          | FIN-201          |

*instructor*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> |
|-----------|-------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       |
| 12121     | Wu          | Finance          |
| 15151     | Mozart      | Music            |

*teaches*

| <i>ID</i> | <i>course_id</i> |
|-----------|------------------|
| 10101     | CS-101           |
| 12121     | FIN-201          |
| 76766     | BIO-101          |

◆ Left Outer Join

*instructor* ⋈<sub>L</sub> *teaches*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>course_id</i> |
|-----------|-------------|------------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       | CS-101           |
| 12121     | Wu          | Finance          | FIN-201          |
| 15151     | Mozart      | Music            | <i>null</i>      |





# Outer Join – Example

## ◆ Right Outer Join

*instructor* ⋈ *teaches*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>course_id</i> |
|-----------|-------------|------------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       | CS-101           |
| 12121     | Wu          | Finance          | FIN-201          |
| 76766     | null        | null             | BIO-101          |

*instructor*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> |
|-----------|-------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       |
| 12121     | Wu          | Finance          |
| 15151     | Mozart      | Music            |

*teaches*

| <i>ID</i> | <i>course_id</i> |
|-----------|------------------|
| 10101     | CS-101           |
| 12121     | FIN-201          |
| 76766     | BIO-101          |

## ◆ Full Outer Join

*instructor* ⋈ *teaches*

| <i>ID</i> | <i>name</i> | <i>dept_name</i> | <i>course_id</i> |
|-----------|-------------|------------------|------------------|
| 10101     | Srinivasan  | Comp. Sci.       | CS-101           |
| 12121     | Wu          | Finance          | FIN-201          |
| 15151     | Mozart      | Music            | null             |
| 76766     | null        | null             | BIO-101          |





# Outer Join using Joins

n Outer join can be expressed using basic operations

| e.g.  $r \bowtie s$  can be written as

$$(r \bowtie s) \cup ((r - \Pi_R(r \bowtie s)) \times \{(null, \dots, null)\})$$

| ID    | name       | dept_name  | course_id |
|-------|------------|------------|-----------|
| 10101 | Srinivasan | Comp. Sci. | CS-101    |
| 12121 | Wu         | Finance    | FIN-201   |

Schema (S - R)

**r**

| ID    | name       | dept_name  |
|-------|------------|------------|
| 10101 | Srinivasan | Comp. Sci. |
| 12121 | Wu         | Finance    |
| 15151 | Mozart     | Music      |

**s**

| ID    | course_id |
|-------|-----------|
| 10101 | CS-101    |
| 12121 | FIN-201   |
| 76766 | BIO-101   |

**$r \bowtie s$**

| ID    | name       | dept_name  | course_id |
|-------|------------|------------|-----------|
| 10101 | Srinivasan | Comp. Sci. | CS-101    |
| 12121 | Wu         | Finance    | FIN-201   |
| 15151 | Mozart     | Music      | null      |



# Null Values

SQL에서...

- n It is possible for tuples to have a null value, denoted by *null*, for some of their attributes
- n *null* signifies an unknown value or that a value does not exist.
- n The result of any arithmetic expression involving *null* is *null*.
- n Aggregate functions simply ignore null values (as in SQL)
- n For duplicate elimination and grouping, null is treated like any other value, and two nulls are assumed to be the same (as in SQL)

- 학년 \* 나이 = ?
- Sum(학년)
- Avg(나이)
- grouping

| 학번 | 이름  | 학과   | 학년   | 나이   | 주소 |
|----|-----|------|------|------|----|
| 1  | 정회훈 | 컴퓨터  | 1    | 21   | 서울 |
| 2  | 김변준 | 컴퓨터  | Null | 22   | 부산 |
| 3  | 이순신 | Null | 3    | 23   | 수원 |
| 4  | 정우진 | 전자   | 4    | Null | 제주 |
| 5  | 김성일 | Null | 2    | 22   | 서울 |



# Null Values

SQL에서...

- n Comparisons with null values return the special truth value: *unknown*
  - | If *false* was used instead of *unknown*, then  $\text{not } (A < 5)$   
would not be equivalent to  $A \geq 5$
- n Three-valued logic using the truth value *unknown*:
  - | OR:  $(\text{unknown or true}) = \text{true},$   
 $(\text{unknown or false}) = \text{unknown}$   
 $(\text{unknown or unknown}) = \text{unknown}$
  - | AND:  $(\text{true and unknown}) = \text{unknown},$   
 $(\text{false and unknown}) = \text{false},$   
 $(\text{unknown and unknown}) = \text{unknown}$
  - | NOT:  $(\text{not unknown}) = \text{unknown}$
  - | In SQL "*P is unknown*" evaluates to true if predicate *P* evaluates to *unknown*
- n Result of select predicate is treated as *false* if it evaluates to *unknown*



# Extended Relational-Algebra-Operations

- n Generalized Projection
- n Aggregate Functions



# Generalized Projection

- n Extends the projection operation by allowing arithmetic functions to be used in the projection list.

$$\Pi_{F_1, F_2, \dots, F_n}(E)$$

- n  $E$  is any relational-algebra expression
- n Each of  $F_1, F_2, \dots, F_n$  are arithmetic expressions involving constants and attributes in the schema of  $E$ .
- n Given relation *instructor*(*ID*, *name*, *dept\_name*, salary) where salary is annual salary, get the same information but with monthly salary

$$\Pi_{ID, name, dept\_name, salary/12}(instructor)$$





# Aggregate Functions and Operations

- n **Aggregation function** takes a collection of values and returns a single value as a result.

**avg**: average value

**min**: minimum value

**max**: maximum value

**sum**: sum of values

**count**: number of values

- n **Aggregate operation** in relational algebra

$$G_1, G_2, \dots, G_n \mathcal{G}_{F_1(A_1), F_2(A_2), \dots, F_n(A_n)}(E)$$

$E$  is any relational-algebra expression

- |  $G_1, G_2, \dots, G_n$  is a list of attributes on which to group (can be empty)
- | Each  $F_i$  is an aggregate function
- | Each  $A_i$  is an attribute name

- n Note: Some books/articles use  $\gamma$  instead of  $\mathcal{G}$  (Calligraphic  $\mathcal{G}$ )



# Aggregate Operation – Example

n Relation  $r$ :

| $A$      | $B$      | $C$ |
|----------|----------|-----|
| $\alpha$ | $\alpha$ | 7   |
| $\alpha$ | $\beta$  | 7   |
| $\beta$  | $\beta$  | 3   |
| $\beta$  | $\beta$  | 10  |

◆  $G_{\text{sum}(c)}(r)$

| $\text{sum}(c)$ |
|-----------------|
| 27              |

◆  $A G_{\text{sum}(c)}(r)$

| $A$      | $\text{sum}(c)$ |
|----------|-----------------|
| $\alpha$ | 14              |
| $\beta$  | 13              |



# Aggregate Operation – Example

- n Find the average salary in each department

*dept\_name*  $\bar{G}$  *avg(salary)* (*instructor*)

| ID    | name       | dept_name  | salary |
|-------|------------|------------|--------|
| 76766 | Crick      | Biology    | 72000  |
| 45565 | Katz       | Comp. Sci. | 75000  |
| 10101 | Srinivasan | Comp. Sci. | 65000  |
| 83821 | Brandt     | Comp. Sci. | 92000  |
| 98345 | Kim        | Elec. Eng. | 80000  |
| 12121 | Wu         | Finance    | 90000  |
| 76543 | Singh      | Finance    | 80000  |
| 32343 | El Said    | History    | 60000  |
| 58583 | Califieri  | History    | 62000  |
| 15151 | Mozart     | Music      | 40000  |
| 33456 | Gold       | Physics    | 87000  |
| 22222 | Einstein   | Physics    | 95000  |

```
select dept_name, avg(salary)
from instructor
group by dept_name
```

| dept_name  | avg(salary) |
|------------|-------------|
| Biology    | 72000       |
| Comp. Sci. | 77333       |
| Elec. Eng. | 80000       |
| Finance    | 85000       |
| History    | 61000       |
| Music      | 40000       |
| Physics    | 91000       |





## Aggregate Functions (Cont.)

- n Result of aggregation does not have a name
  - | Can use rename operation to give it a name
  - | For convenience, we permit renaming as part of aggregate operation

*dept\_name*  $\mathcal{G}$  *avg(salary) as avg\_sal (instructor)*

| <i>dept_name</i> | <i>avg_salary</i> |
|------------------|-------------------|
| Biology          | 72000             |
| Comp. Sci.       | 77333             |
| Elec. Eng.       | 80000             |
| Finance          | 85000             |
| History          | 61000             |
| Music            | 40000             |
| Physics          | 91000             |





# SQL and Relational Algebra

n **select**  $A_1, A_2, \dots, A_n$   
**from**  $r_1, r_2, \dots, r_m$   
**where**  $P$

is equivalent to the following expression in multiset relational algebra

$$\Pi_{A_1, \dots, A_n} (\sigma_P (r_1 \times r_2 \times \dots \times r_m))$$

n **select**  $A_1, A_2, \text{sum}(A_3)$   
**from**  $r_1, r_2, \dots, r_m$   
**where**  $P$   
**group by**  $A_1, A_2$

is equivalent to the following expression in multiset relational algebra

$$A_1, A_2 \mathcal{G} \text{sum}(A_3) (\sigma_P (r_1 \times r_2 \times \dots \times r_m))$$





The End.



Any Question ?

Database System Concepts, 6<sup>th</sup> Ed.

©Silberschatz, Korth and Sudarshan (revised by Sangwoo Park)

See [www.db-book.com](http://www.db-book.com) for conditions on re-use



# University Database

## instructor

| ID    | name       | dept_name  | salary |
|-------|------------|------------|--------|
| 10101 | Srinivasan | Comp. Sci. | 65000  |
| 12121 | Wu         | Finance    | 90000  |
| 15151 | Mozart     | Music      | 40000  |
| 22222 | Einstein   | Physics    | 95000  |
| 32343 | El Said    | History    | 60000  |
| 33456 | Gold       | Physics    | 87000  |
| 45565 | Katz       | Comp. Sci. | 75000  |
| 58583 | Califieri  | History    | 62000  |
| 76543 | Singh      | Finance    | 80000  |
| 76766 | Crick      | Biology    | 72000  |
| 83821 | Brandt     | Comp. Sci. | 92000  |
| 98345 | Kim        | Elec. Eng. | 80000  |

## course

| course_id | title                      | dept_name  | credits |
|-----------|----------------------------|------------|---------|
| BIO-101   | Intro. to Biology          | Biology    | 4       |
| BIO-301   | Genetics                   | Biology    | 4       |
| BIO-399   | Computational Biology      | Biology    | 3       |
| CS-101    | Intro. to Computer Science | Comp. Sci. | 4       |
| CS-190    | Game Design                | Comp. Sci. | 4       |
| CS-315    | Robotics                   | Comp. Sci. | 3       |
| CS-319    | Image Processing           | Comp. Sci. | 3       |
| CS-347    | Database System Concepts   | Comp. Sci. | 3       |
| EE-181    | Intro. to Digital Systems  | Elec. Eng. | 3       |
| FIN-201   | Investment Banking         | Finance    | 3       |
| HIS-351   | World History              | History    | 3       |
| MU-199    | Music Video Production     | Music      | 3       |
| PHY-101   | Physical Principles        | Physics    | 4       |

## prereq

| course_id | prereq_id |
|-----------|-----------|
| BIO-301   | BIO-101   |
| BIO-399   | BIO-101   |
| CS-190    | CS-101    |
| CS-315    | CS-101    |
| CS-319    | CS-101    |
| CS-347    | CS-101    |
| EE-181    | PHY-101   |

## department

| dept_name  | building | budget |
|------------|----------|--------|
| Biology    | Watson   | 90000  |
| Comp. Sci. | Taylor   | 100000 |
| Elec. Eng. | Taylor   | 85000  |
| Finance    | Painter  | 120000 |
| History    | Painter  | 50000  |
| Music      | Packard  | 80000  |
| Physics    | Watson   | 70000  |

## section

| course_id | sec_id | semester | year | building | room_number | time_slot_id |
|-----------|--------|----------|------|----------|-------------|--------------|
| BIO-101   | 1      | Summer   | 2009 | Painter  | 514         | B            |
| BIO-301   | 1      | Summer   | 2010 | Painter  | 514         | A            |
| CS-101    | 1      | Fall     | 2009 | Packard  | 101         | H            |
| CS-101    | 1      | Spring   | 2010 | Packard  | 101         | F            |
| CS-190    | 1      | Spring   | 2009 | Taylor   | 3128        | E            |
| CS-190    | 2      | Spring   | 2009 | Taylor   | 3128        | A            |
| CS-315    | 1      | Spring   | 2010 | Watson   | 120         | D            |
| CS-319    | 1      | Spring   | 2010 | Watson   | 100         | B            |
| CS-319    | 2      | Spring   | 2010 | Taylor   | 3128        | C            |
| CS-347    | 1      | Fall     | 2009 | Taylor   | 3128        | A            |
| EE-181    | 1      | Spring   | 2009 | Taylor   | 3128        | C            |
| FIN-201   | 1      | Spring   | 2010 | Packard  | 101         | B            |
| HIS-351   | 1      | Spring   | 2010 | Painter  | 514         | C            |
| MU-199    | 1      | Spring   | 2010 | Packard  | 101         | D            |
| PHY-101   | 1      | Fall     | 2009 | Watson   | 100         | A            |

## teaches

| ID    | course_id | sec_id | semester | year |
|-------|-----------|--------|----------|------|
| 10101 | CS-101    | 1      | Fall     | 2009 |
| 10101 | CS-315    | 1      | Spring   | 2010 |
| 10101 | CS-347    | 1      | Fall     | 2009 |
| 12121 | FIN-201   | 1      | Spring   | 2010 |
| 15151 | MU-199    | 1      | Spring   | 2010 |
| 22222 | PHY-101   | 1      | Fall     | 2009 |
| 32343 | HIS-351   | 1      | Spring   | 2010 |
| 45565 | CS-101    | 1      | Spring   | 2010 |
| 45565 | CS-319    | 1      | Spring   | 2010 |
| 76766 | BIO-101   | 1      | Summer   | 2009 |
| 76766 | BIO-301   | 1      | Summer   | 2010 |
| 83821 | CS-190    | 1      | Spring   | 2009 |
| 83821 | CS-190    | 2      | Spring   | 2009 |
| 83821 | CS-319    | 2      | Spring   | 2010 |
| 98345 | EE-181    | 1      | Spring   | 2009 |





# Banking Database

*branch*

| <i>branch_name</i> | <i>branch_city</i> | <i>assets</i> |
|--------------------|--------------------|---------------|
| Brighton           | Brooklyn           | 7100000       |
| Downtown           | Brooklyn           | 9000000       |
| Mianus             | Horseneck          | 400000        |
| North Town         | Rye                | 3700000       |
| Perryridge         | Horseneck          | 1700000       |
| Pownal             | Bennington         | 300000        |
| Redwood            | Palo Alto          | 2100000       |
| Round Hill         | Horseneck          | 8000000       |

*customer*

| <i>customer_name</i> | <i>customer_street</i> | <i>customer_city</i> |
|----------------------|------------------------|----------------------|
| Adams                | Spring                 | Pittsfield           |
| Brooks               | Senator                | Brooklyn             |
| Curry                | North                  | Rye                  |
| Glenn                | Sand Hill              | Woodside             |
| Green                | Walnut                 | Stamford             |
| Hayes                | Main                   | Harrison             |
| Johnson              | Alma                   | Palo Alto            |
| Jones                | Main                   | Harrison             |
| Lindsay              | Park                   | Pittsfield           |
| Smith                | North                  | Rye                  |
| Turner               | Putnam                 | Stamford             |
| Williams             | Nassau                 | Princeton            |

*borrower*

| <i>customer_name</i> | <i>loan_number</i> |
|----------------------|--------------------|
| Adams                | L-16               |
| Curry                | L-93               |
| Hayes                | L-15               |
| Jackson              | L-14               |
| Jones                | L-17               |
| Smith                | L-11               |
| Smith                | L-23               |
| Williams             | L-17               |

*loan*

| <i>loan_number</i> | <i>branch_name</i> | <i>amount</i> |
|--------------------|--------------------|---------------|
| L-11               | Round Hill         | 900           |
| L-14               | Downtown           | 1500          |
| L-15               | Perryridge         | 1500          |
| L-16               | Perryridge         | 1300          |
| L-17               | Downtown           | 1000          |
| L-23               | Redwood            | 2000          |
| L-93               | Mianus             | 500           |

*depositor*

| <i>customer_name</i> | <i>account_number</i> |
|----------------------|-----------------------|
| Hayes                | A-102                 |
| Johnson              | A-101                 |
| Johnson              | A-201                 |
| Jones                | A-217                 |
| Lindsay              | A-222                 |
| Smith                | A-215                 |
| Turner               | A-305                 |

*account*

| <i>account_number</i> | <i>branch_name</i> | <i>balance</i> |
|-----------------------|--------------------|----------------|
| A-101                 | Downtown           | 500            |
| A-215                 | Mianus             | 700            |
| A-102                 | Perryridge         | 400            |
| A-305                 | Round Hill         | 350            |
| A-201                 | Brighton           | 900            |
| A-222                 | Redwood            | 700            |
| A-217                 | Brighton           | 750            |





## ***instructor x teaches***

Fig 3.6

| <i>Inst.ID</i> | <i>name</i> | <i>dept_name</i> | <i>salary</i> | <i>teaches.ID</i> | <i>course_id</i> | <i>sec_id</i> | <i>semester</i> | <i>year</i> |
|----------------|-------------|------------------|---------------|-------------------|------------------|---------------|-----------------|-------------|
| 10101          | Srinivasan  | Physics          | 95000         | 10101             | CS-101           | 1             | Fall            | 2009        |
| 10101          | Srinivasan  | Physics          | 95000         | 10101             | CS-315           | 1             | Spring          | 2010        |
| 10101          | Srinivasan  | Physics          | 95000         | 10101             | CS-347           | 1             | Fall            | 2009        |
| 10101          | Srinivasan  | Physics          | 95000         | 12121             | FIN-201          | 1             | Spring          | 2010        |
| 10101          | Srinivasan  | Physics          | 95000         | 15151             | MU-199           | 1             | Spring          | 2010        |
| 10101          | Srinivasan  | Physics          | 95000         | 22222             | PHY-101          | 1             | Fall            | 2009        |
| ...            | ...         | ...              | ...           | ...               | ...              | ...           | ...             | ...         |
| ...            | ...         | ...              | ...           | ...               | ...              | ...           | ...             | ...         |
| 12121          | Wu          | Physics          | 95000         | 10101             | CS-101           | 1             | Fall            | 2009        |
| 12121          | Wu          | Physics          | 95000         | 10101             | CS-315           | 1             | Spring          | 2010        |
| 12121          | Wu          | Physics          | 95000         | 10101             | CS-347           | 1             | Fall            | 2009        |
| 12121          | Wu          | Physics          | 95000         | 10101             | FIN-201          | 1             | Spring          | 2010        |
| 12121          | Wu          | Physics          | 95000         | 15151             | MU-199           | 1             | Spring          | 2010        |
| 12121          | Wu          | Physics          | 95000         | 22222             | PHY-101          | 1             | Fall            | 2009        |
| ...            | ...         | ...              | ...           | ...               | ...              | ...           | ...             | ...         |
| ...            | ...         | ...              | ...           | ...               | ...              | ...           | ...             | ...         |





# Chapter 6: Relational Algebra Expression : Practices

Database System Concepts, 6<sup>th</sup> Ed.

©Silberschatz, Korth and Sudarshan (revised by Sangwoo Park)

See [www.db-book.com](http://www.db-book.com) for conditions on re-use



# Banking Example

## Banking Database

- n *branch* (*branch\_name*, *branch\_city*, *assets*)
- n *customer* (*customer\_name*, *customer\_street*, *customer\_city*)
- n *account* (*account\_number*, *branch\_name*, *balance*)
- n *loan* (*loan\_number*, *branch\_name*, *amount*)
- n *depositor* (*customer\_name*, *account\_number*)
- n *borrower* (*customer\_name*, *loan\_number*)





# Query principles (1)

- ◆ Find the names of all customers who live in Palo Alto (city)

②

| <i>customer_name</i> | <i>customer_street</i> | <i>customer_city</i> |
|----------------------|------------------------|----------------------|
| Adams                | Spring                 | Pittsfield           |
| Brooks               | Senator                | Brooklyn             |
| Curry                | North                  | Rye                  |
| Glenn                | Sand Hill              | Woodside             |
| Green                | Walnut                 | Stamford             |
| Hayes                | Main                   | Harrison             |
| Johnson              | Alma                   | Palo Alto            |
| Jones                | Main                   | Harrison             |
| Lindsay              | Park                   | Pittsfield           |
| Smith                | North                  | Rye                  |
| Turner               | Putnam                 | Stamford             |
| Williams             | Nassau                 | Princeton            |

①

*customer*



## Query principles (2)

- ◆ Find the account\_numbers of accounts held by customers who live in Palo Alto (city)

*customer*

| <i>customer_name</i> | <i>customer_street</i> | <i>customer_city</i> |
|----------------------|------------------------|----------------------|
| Adams                | Spring                 | Pittsfield           |
| Brooks               | Senator                | Brooklyn             |
| Curry                | North                  | Rye                  |
| Glenn                | Sand Hill              | Woodside             |
| Green                | Walnut                 | Stamford             |
| Hayes                | Main                   | Harrison             |
| Johnson              | Alma                   | Palo Alto            |
| Jones                | Main                   | Harrison             |
| Lindsay              | Park                   | Pittsfield           |
| Smith                | North                  | Rye                  |
| Turner               | Putnam                 | Stamford             |
| Williams             | Nassau                 | Princeton            |

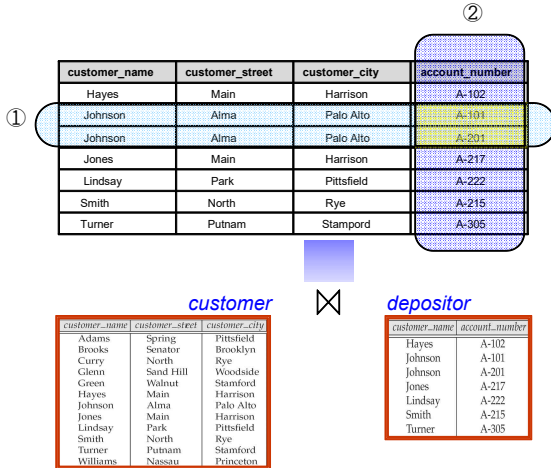
*depositor*

| <i>customer_name</i> | <i>account_number</i> |
|----------------------|-----------------------|
| Hayes                | A-102                 |
| Johnson              | A-101                 |
| Johnson              | A-201                 |
| Jones                | A-217                 |
| Lindsay              | A-222                 |
| Smith                | A-215                 |
| Turner               | A-305                 |



## Query principles (2)

- ◆ Find the account\_numbers of accounts held by customers who live in Palo Alto (city)





# Example Queries

- n Find all loans of over \$1200

$$\sigma_{amount > 1200} (loan)$$

- ◆ Find the loan number for each loan of an amount greater than \$1200

$$\Pi_{loan\_number} (\sigma_{amount > 1200} (loan))$$



## Example Queries

- n Find the names of all customers who have a loan, an account, or both, from the bank

$$\Pi_{customer\_name}(borrower) \cup \Pi_{customer\_name}(depositor)$$

- ♦ Find the names of all customers who have a loan and an account at bank.

$$\Pi_{customer\_name}(borrower) \cap \Pi_{customer\_name}(depositor)$$



# Example Queries

- n Find the names of all customers who have a loan at the Perryridge branch.

- Query 1

$\Pi_{\text{customer\_name}} (\sigma_{\text{branch\_name} = \text{"Perryridge"}} ($   
 $\sigma_{\text{borrower.loan\_number} = \text{loan.loan\_number}} (\text{borrower} \times \text{loan}))$

loan = 10,000 tuples

그중 Perryridge loan = 100

tuples

borrower = 20,000 tuples

$20,000 \times 10,000 = 200,000,000$

- Query 2

$\Pi_{\text{customer\_name}} (\sigma_{\text{loan.loan\_number} = \text{borrower.loan\_number}} ($   
 $\sigma_{\text{branch\_name} = \text{"Perryridge"}} (\text{loan})) \times \text{borrower})$

$10,000 \rightarrow 100$

$20,000 \times 100 = 2,000,000$

2억 → 200만



## Example Queries

- ◆ Find the names of all customers who have a loan at the Perryridge branch but do not have an account at any branch of the bank.

$\Pi_{customer\_name} (\sigma_{branch\_name = "Perryridge"} ($   
 $\sigma_{borrower.loan\_number = loan.loan\_number} (borrower \times loan))) -$   
 $\Pi_{customer\_name} (depositor)$



# Example Queries

## n Find the largest account balance

| Strategy:

‣ Find those balances that are *not* the largest

— Rename *account* relation as *d* so that we can compare each account balance with all others

‣ Use set difference to find those account balances that were *not* found in the earlier step.

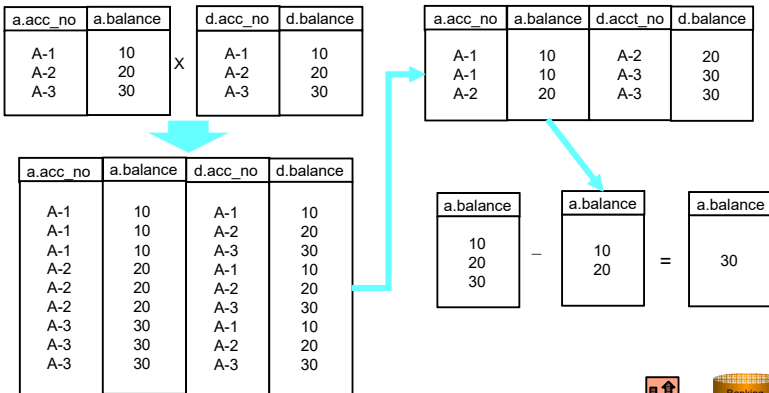
| The query is:

$$\Pi_{balance}(account) - \Pi_{account.balance}(\sigma_{account.balance < d.balance}(account \times \rho_d(account)))$$



## Example Queries

$$\Pi_{balance}(account) - \Pi_{account.balance}(\sigma_{account.balance < d.balance}(account \times \rho_d(account)))$$





# Bank Example Queries

- n Find the name of all customers who have a loan at the bank , the loan-number, and the amount of the loan

$\Pi_{customer-name, loan-number, amount}(borrower \bowtie loan)$



# Bank Example Queries

- n Find all customers who have an account from at least the “Downtown” and the “Uptown” branches.

$$\Pi_{customer\_name} (\sigma_{branch\_name = \text{“Downtown”}} (depositor \bowtie_{account} account)) \cap \Pi_{customer\_name} (\sigma_{branch\_name = \text{“Uptown”}} (depositor \bowtie_{account} account))$$